SISTEMAS DISTRIBUIDOS

Evaluación-taller: Agregador resiliente de APIs

Escuela de Ingeniería de Sistemas e Informática — Universidad Industrial de Santander

1. Datos generales y propósito

Cada grupo construye en FastAPI un servicio agregador que consume una API externa elegida de un catálogo y un servicio inestable que el propio grupo ejecuta en sus máquinas, y expone su propia API resiliente.

- **Modalidad:** evaluación-taller en grupos de máximo 4 personas.
- **Dedicación prevista:** carga calculada para una sesión de clase de 2 horas más trabajo autónomo, distribuidos como el grupo prefiera.
- **Referencia:** Vitillo, Understanding Distributed Systems, capítulo de APIs y capítulos de resiliencia (timeouts, reintentos, idempotencia, caché).
- **Tecnologías:** Python, FastAPI, Uvicorn, httpx, Pydantic v2.

Al terminar, el grupo debe poder demostrar que:

1. Consume endpoints de una API externa encadenando al menos dos llamadas.
2. Diseña y documenta una API propia con recursos, códigos de estado y contrato OpenAPI coherentes.
3. Tolera fallos de una dependencia remota con timeouts, reintentos selectivos y backoff con jitter.
4. Reduce latencia y carga sobre la dependencia mediante caché.
5. Garantiza que una operación de escritura repetida no produzca efectos duplicados.

2. Organización

El grupo distribuye libremente su tiempo entre la clase y el trabajo autónomo: no hay partes asignadas a uno u otro momento ni tiempos por actividad. El docente acompaña el avance durante la sesión de clase.

Grupos. Máximo 4 integrantes. Cada grupo elige una API del catálogo. Se sugiere que dos grupos no elijan la misma API, para facilitar la revisión y el consumo cruzado opcional.

Roles sugeridos (no obligatorios). Repartir el trabajo así evita que una sola persona escriba todo el código:

- Operación del servicio inestable: lo arranca en su equipo, ajusta sus parámetros y comparte la dirección con el grupo.
- Cliente de la API externa.
- Resiliencia frente al servicio inestable.
- API propia, modelos Pydantic, pruebas y README.

3. Catálogo de APIs externas

Cada grupo elige una de estas APIs públicas; ninguna exige clave en su uso básico y todas permiten encadenar dos o más endpoints. El requisito mínimo es que el endpoint propio combine al menos dos llamadas distintas a la API elegida.

API	Endpoints para encadenar	Idea de endpoint propio	Lo que obliga a manejar
Open-Meteo open-meteo.com	Geocodificación por nombre → pronóstico por coordenadas	`GET /v1/ciudades/{nombre}/clima`	Parámetros de consulta, JSON anidado, ciudad inexistente
Frankfurter frankfurter.dev	`/v1/currencies` → `/v1/latest?base=…&symbols=…` → series por rango de fechas	`GET /v1/conversiones?de=USD&a=EUR&monto=100`	Validación de monedas, fechas sin publicación. No incluye COP
Nager.Date date.nager.at	Países disponibles → festivos por año y país → fines de semana largos	`GET /v1/paises/{codigo}/puentes/{anio}`	Códigos de país inválidos, listas largas
PokeAPI pokeapi.co	Listado paginado (`limit`, `offset`) → detalle → tipo	`GET /v1/equipos/{tipo}/resumen`	Paginación, muchas llamadas por petición, uso responsable
Open Library openlibrary.org	Búsqueda → obra → autor	`GET /v1/autores/{nombre}/obras`	Campos faltantes, respuestas grandes
datos.gov.co (API SODA)	Consulta SoQL con `$where`, `$limit`, `$offset` sobre un dataset abierto	Resumen filtrado de un dataset colombiano	Límite de peticiones bajo sin token; se recomienda un App Token gratuito

Un grupo puede proponer otra API pública sin clave, con aprobación previa del docente.

> **Importante.** Todos los equipos de la sala salen a internet con la misma IP pública, de modo que los límites de peticiones de las APIs se reparten entre todos los grupos. Esto hace que la caché de la Parte C sea una necesidad real, no un adorno.

4. Servicio inestable

El docente entrega el archivo `servicio_inestable.py` (FastAPI, código completo en el Anexo), que simula un registro de sensores y falla a propósito. El propio grupo lo ejecuta en sus máquinas: uno de los integrantes puede arrancarlo con `uvicorn` y compartir la dirección, o cada integrante puede correrlo en `localhost` mientras desarrolla.

Arranque

```
python -m venv venv
venv\Scripts\activate            # Windows  (Linux: source venv/bin/activate)
pip install fastapi uvicorn httpx

python servicio_inestable.py                          # valores por defecto, puerto 9000
python servicio_inestable.py --tasa-fallos 0.4 --semilla 42
python servicio_inestable.py --host 0.0.0.0           # accesible desde otros equipos
```

Comportamiento que deben esperar

- Respuestas 503 aleatorias.
- Respuestas lentas, de varios segundos.
- Respuestas 429 con la cabecera `Retry-After`. Si el cliente vuelve a llamar antes de ese tiempo, recibe otro 429.
- Resultados paginados con un cursor `siguiente`.
- Errores 400 y 404 legítimos, que **no** deben reintentarse.

Endpoints

Método y ruta	Uso
`GET /sensores?cursor=…`	Lista paginada de sensores; el campo `siguiente` es `null` en la última página
`GET /sensores/{id}/lectura`	Lectura actual de un sensor (404 si no existe)
`GET /salud`	Comprobación de vida del servicio; nunca falla
`GET /config`	Parámetros con los que se arrancó el servicio

Parámetros de arranque

Opción	Por defecto	Efecto
`--tasa-fallos`	0.30	Probabilidad de responder 503
`--tasa-429`	0.10	Probabilidad de responder 429 con Retry-After
`--tasa-lentitud`	0.20	Probabilidad de que una respuesta sea lenta
`--latencia`	4.0	Segundos de espera en las respuestas lentas
`--retry-after`	2	Segundos indicados en la cabecera Retry-After
`--semilla`	ninguna	Semilla aleatoria para hacer reproducibles los fallos
`--tam-pagina`	5	Sensores por página
`--host` / `--puerto`	127.0.0.1 / 9000	Interfaz y puerto de escucha

El agregador debe leer la URL del servicio inestable desde una variable de entorno, porque el docente revisará cada entrega contra su propia instancia con otros parámetros.

> **Sala Windows.** Si el firewall impide que otros equipos se conecten al servicio, cada integrante lo ejecuta en `localhost`; para eso no se necesitan permisos de administrador ni reiniciar el equipo.

5. Actividades

Las partes siguen un orden sugerido de trabajo, pero el grupo decide cuándo hace cada una. La Parte E es opcional.

Parte A — Consumir la API externa

1. Crear un entorno virtual e instalar `fastapi`, `uvicorn` y `httpx` con `pip`. No se requiere instalar nada que exija reiniciar el equipo.
2. Arrancar el servicio inestable en uno o varios equipos del grupo y comprobar que responde en `/salud`.
3. Explorar la API elegida con el navegador o `curl` e identificar los endpoints que se van a encadenar.
4. Escribir un cliente con `httpx` que haga las llamadas encadenadas.
5. Definir modelos Pydantic v2 para las respuestas y rechazar con un error claro las que no cumplan el modelo.
6. Fijar un timeout explícito en cada llamada saliente.

Parte B — Exponer la API propia

1. Diseñar al menos un endpoint `GET` que combine los datos de la API externa.
2. Usar códigos de estado coherentes: 200 si todo sale bien, 404 si el recurso no existe en la API externa, 422 para parámetros inválidos, 502 o 504 si la dependencia falla. No devolver un 500 genérico.
3. Agregar un endpoint `GET /salud` propio.
4. Revisar que `/docs` describa correctamente el contrato.

Parte C — Resiliencia y caché

1. Integrar el servicio inestable en un endpoint propio, leyendo su URL desde una variable de entorno.
2. Implementar reintentos con backoff exponencial y jitter, con un número máximo de intentos.
3. Reintentar solo lo reintentable: 503, 429 y timeouts sí; 400 y 404 no.
4. Respetar `Retry-After` cuando venga en la respuesta.
5. Recorrer la paginación completa del servicio inestable.
6. Agregar caché en memoria con TTL para la API externa.
7. Medir la latencia del endpoint propio con caché y sin ella, y registrar los resultados.

Parte D — Idempotencia y análisis

1. Implementar `POST /v1/alertas` con la cabecera `Idempotency-Key`: dos peticiones con la misma clave crean una sola alerta y devuelven la misma respuesta.
2. Demostrarlo con dos peticiones repetidas y su resultado.
3. Responder en el README las preguntas de análisis:
   a) ¿Por qué es peligroso reintentar un `POST` sin clave de idempotencia?
   b) ¿Qué ocurriría si 50 clientes reintentaran al mismo tiempo sin jitter?
   c) ¿Qué tan desactualizados pueden estar los datos que entrega su caché, y cuándo sería inaceptable?
   d) ¿Qué cambiaría en su diseño si el servicio corriera en varias réplicas detrás de un balanceador?

Parte E — Consumo cruzado (opcional, bonificación)

El grupo consume un endpoint de otro grupo usando únicamente su documentación OpenAPI, y reporta en el README qué le faltó o le sobró al contrato.

6. Entregables

Una sola entrega por grupo en Moodle, en un archivo comprimido.

- Código fuente del agregador y un archivo `requirements.txt`. La URL del servicio inestable se lee desde una variable de entorno.
- `README.md` con: integrantes y rol de cada uno, API elegida, instrucciones para arrancar el servicio inestable y el agregador, tabla de latencias con caché y sin ella, demostración de idempotencia y respuestas a las preguntas de la Parte D.
- Evidencias de los reintentos: registros del servicio o capturas de `tcpspy.py` donde se vean los intentos, las esperas y el respeto de `Retry-After`.
- Una colección de peticiones de prueba (`curl`, archivo `.http` o script en Python) que el docente pueda ejecutar para revisar cada endpoint.

7. Evaluación

La nota es grupal sobre 100 puntos, más hasta 10 puntos de bonificación por la Parte E. El docente revisa la entrega ejecutando la colección de peticiones contra su propia instancia del servicio inestable.

Criterio	Puntos
Consumo de la API externa: llamadas encadenadas, modelos, timeouts	15
Diseño de la API propia: recursos, códigos de estado, contrato OpenAPI	20
Resiliencia: reintentos selectivos, backoff con jitter, `Retry-After`, paginación	30
Caché con TTL y medición de latencia	10
Idempotencia demostrada	10
Preguntas de análisis	15
Total	**100**
Consumo cruzado (bonificación)	+10

El docente puede pedir a cualquier integrante que explique una parte del código, por ejemplo mientras acompaña el avance en clase.

Anexo. Código de servicio_inestable.py

El mismo archivo se entrega por separado en Moodle. No es necesario modificarlo; se configura con los parámetros de arranque.

```
"""
servicio_inestable.py — Registro de sensores que falla a propósito.

Sistemas Distribuidos · Evaluación-taller "Agregador resiliente de APIs"
Escuela de Ingeniería de Sistemas e Informática — UIS

Uso:
    pip install fastapi uvicorn
    python servicio_inestable.py                      # valores por defecto
    python servicio_inestable.py --tasa-fallos 0.4 --latencia 5 --semilla 42
    python servicio_inestable.py --host 0.0.0.0       # accesible desde otros equipos

Endpoints:
    GET /salud                    Comprobación de vida (nunca falla)
    GET /config                   Parámetros con los que se arrancó el servicio
    GET /sensores?cursor=...      Lista paginada de sensores
    GET /sensores/{id}/lectura    Lectura actual de un sensor

Fallos inyectados (solo en /sensores...):
    - 429 Too Many Requests con cabecera Retry-After. Si el cliente vuelve a
      llamar antes de que pase ese tiempo, recibe otro 429.
    - 503 Service Unavailable aleatorios.
    - Respuestas lentas (se duerme --latencia segundos antes de responder).
Errores legítimos (NO deben reintentarse):
    - 400 si el cursor es inválido.
    - 404 si el sensor no existe.
"""

import argparse
import asyncio
import base64
import random
import time
from datetime import datetime, timezone
from typing import Dict, List, Optional

import uvicorn
from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import JSONResponse

# ---------------------------------------------------------------------------
# Configuración por línea de órdenes
# ---------------------------------------------------------------------------

parser = argparse.ArgumentParser(description="Servicio de sensores inestable")
parser.add_argument("--host", default="127.0.0.1",
                    help="Interfaz de escucha (0.0.0.0 para aceptar otros equipos)")
parser.add_argument("--puerto", type=int, default=9000)
parser.add_argument("--tasa-fallos", type=float, default=0.30,
                    help="Probabilidad de responder 503 (0 a 1)")
parser.add_argument("--tasa-429", type=float, default=0.10,
                    help="Probabilidad de responder 429 con Retry-After (0 a 1)")
parser.add_argument("--tasa-lentitud", type=float, default=0.20,
                    help="Probabilidad de que una respuesta sea lenta (0 a 1)")
```

```
parser.add_argument("--latencia", type=float, default=4.0,
                    help="Segundos de espera en las respuestas lentas")
parser.add_argument("--retry-after", type=int, default=2,
                    help="Segundos indicados en la cabecera Retry-After")
parser.add_argument("--semilla", type=int, default=None,
                    help="Semilla aleatoria para hacer reproducibles los fallos")
parser.add_argument("--tam-pagina", type=int, default=5,
                    help="Sensores por página")
parser.add_argument("--num-sensores", type=int, default=23)
ARGS = parser.parse_args()

if ARGS.tasa_fallos + ARGS.tasa_429 > 1:
    parser.error("--tasa-fallos + --tasa-429 no puede superar 1")

rng = random.Random(ARGS.semilla)

# ---------------------------------------------------------------------------
# Datos simulados
# ---------------------------------------------------------------------------

TIPOS = {
    "temperatura": ("°C", 18.0, 32.0),
    "humedad": ("%", 40.0, 95.0),
    "ruido": ("dB", 35.0, 90.0),
    "co2": ("ppm", 400.0, 1500.0),
}
UBICACIONES = ["Edificio A", "Edificio B", "Laboratorio 1", "Laboratorio 2",
               "Biblioteca", "Cafetería", "Auditorio"]

_datos = random.Random(ARGS.semilla)
SENSORES: List[Dict] = []
for i in range(1, ARGS.num_sensores + 1):
    tipo = _datos.choice(list(TIPOS))
    SENSORES.append({
        "id": f"S-{i:03d}",
        "tipo": tipo,
        "ubicacion": _datos.choice(UBICACIONES),
    })
POR_ID = {s["id"]: s for s in SENSORES}

# Bloqueos por cliente tras un 429: ip -> instante en que se levanta el bloqueo
bloqueados: Dict[str, float] = {}

app = FastAPI(
    title="Servicio inestable de sensores",
    description="Dependencia poco fiable para practicar timeouts, reintentos "
                "y respeto de Retry-After.",
    version="1.0",
)


def log(ip: str, metodo: str, ruta: str, resultado: str) -> None:
    hora = datetime.now().strftime("%H:%M:%S.%f")[:-3]
    print(f"[{hora}] {ip:>15} {metodo} {ruta} -> {resultado}", flush=True)


# ---------------------------------------------------------------------------
# Inyección de fallos
# ---------------------------------------------------------------------------
```

```
@app.middleware("http")
async def inyectar_fallos(request: Request, call_next):
    ruta = request.url.path
    ip = request.client.host if request.client else "desconocido"
    metodo = request.method

    if not ruta.startswith("/sensores"):
        return await call_next(request)

    ahora = time.monotonic()

    # 1. Cliente que no respetó un Retry-After anterior
    hasta = bloqueados.get(ip)
    if hasta is not None and ahora < hasta:
        restante = max(1, int(hasta - ahora + 0.999))
        log(ip, metodo, ruta, f"429 (no respetó Retry-After, faltan {restante} s)")
        return JSONResponse(
            status_code=429,
            content={"detalle": "Demasiadas peticiones; respete Retry-After"},
            headers={"Retry-After": str(restante)},
        )

    sorteo = rng.random()

    # 2. Límite de peticiones simulado
    if sorteo < ARGS.tasa_429:
        bloqueados[ip] = ahora + ARGS.retry_after
        log(ip, metodo, ruta, f"429 (Retry-After: {ARGS.retry_after})")
        return JSONResponse(
            status_code=429,
            content={"detalle": "Demasiadas peticiones"},
            headers={"Retry-After": str(ARGS.retry_after)},
        )

    # 3. Fallo transitorio del servidor
    if sorteo < ARGS.tasa_429 + ARGS.tasa_fallos:
        log(ip, metodo, ruta, "503")
        return JSONResponse(
            status_code=503,
            content={"detalle": "Servicio no disponible temporalmente"},
        )

    # 4. Respuesta lenta
    if rng.random() < ARGS.tasa_lentitud:
        log(ip, metodo, ruta, f"lenta ({ARGS.latencia} s)")
        await asyncio.sleep(ARGS.latencia)

    respuesta = await call_next(request)
    log(ip, metodo, ruta, str(respuesta.status_code))
    return respuesta


# ---------------------------------------------------------------------------
# Cursores opacos para la paginación
# ---------------------------------------------------------------------------

def codificar_cursor(desplazamiento: int) -> str:
    return base64.urlsafe_b64encode(f"o:{desplazamiento}".encode()).decode()
```

```
def decodificar_cursor(cursor: str) -> int:
    try:
        texto = base64.urlsafe_b64decode(cursor.encode()).decode()
        prefijo, valor = texto.split(":")
        desplazamiento = int(valor)
        if prefijo != "o" or not 0 <= desplazamiento < len(SENSORES):
            raise ValueError
        return desplazamiento
    except Exception:
        raise HTTPException(status_code=400, detail="Cursor inválido")


# ---------------------------------------------------------------------------
# Endpoints
# ---------------------------------------------------------------------------

@app.get("/salud")
def salud():
    return {"estado": "ok"}


@app.get("/config")
def config():
    return {
        "tasa_fallos": ARGS.tasa_fallos,
        "tasa_429": ARGS.tasa_429,
        "tasa_lentitud": ARGS.tasa_lentitud,
        "latencia_s": ARGS.latencia,
        "retry_after_s": ARGS.retry_after,
        "semilla": ARGS.semilla,
        "tam_pagina": ARGS.tam_pagina,
        "num_sensores": len(SENSORES),
    }


@app.get("/sensores")
def listar_sensores(cursor: Optional[str] = None):
    inicio = 0 if cursor is None else decodificar_cursor(cursor)
    fin = inicio + ARGS.tam_pagina
    siguiente = codificar_cursor(fin) if fin < len(SENSORES) else None
    return {"sensores": SENSORES[inicio:fin], "siguiente": siguiente}


@app.get("/sensores/{sensor_id}/lectura")
def lectura(sensor_id: str):
    sensor = POR_ID.get(sensor_id)
    if sensor is None:
        raise HTTPException(status_code=404, detail=f"No existe el sensor {sensor_id}")
    unidad, minimo, maximo = TIPOS[sensor["tipo"]]
    return {
        "sensor_id": sensor_id,
        "tipo": sensor["tipo"],
        "valor": round(random.uniform(minimo, maximo), 1),
        "unidad": unidad,
        "marca_tiempo": datetime.now(timezone.utc).isoformat(timespec="seconds"),
    }
```

```
if __name__ == "__main__":
    print(f"Servicio inestable en http://{ARGS.host}:{ARGS.puerto}  "
          f"(documentación en /docs)")
    print(f"Parámetros: {config()}")
    uvicorn.run(app, host=ARGS.host, port=ARGS.puerto, log_level="warning")
```